Architecture or missing logic? And are the tests hard enough? (research note, 2026-10-24)

NO CODE CHANGED. Research only; this proposes the next experiments.

The two questions

1. Are the differences we see the architectures' own capability, or logic (handles) that we have not found yet?
2. Are the 12 tasks hard and varied enough to tell architectures apart?

What the field does for question 1

The standard method is **oracle substitution with a shared stack**: replace one module with a perfect version, keep everything else the same, and see how much the score rises. The module whose oracle gives the biggest jump is the bottleneck.

Paper	Mechanism	What we take
PRISM (2026)	Perception, memory and planning probes that can be kept, replaced or bypassed; oracle perception as the upper bound, then controlled noise	Per-level oracle swaps; optionally graded degradation
StructNav (2023)	Ground-truth odometry, segmentation and scene graph swapped in one at a time, with the same planner; unexplored objects masked so the oracle is not handed the answer	One swap at a time; the oracle must not give away the task answer
OPEx (ACL 2024)	Observer / planner / executor split; perception and low-level execution were the bottlenecks, not the LLM planner	Expect the limit outside the planner, and measure it
CanTest (2026)	Capability oracles label each failure as perception, memory, planning or decision	A failure label per flight
Embodied Agent Interface (NeurIPS 2024)	Per-module scores; a single success rate hides where models fail	Per-level scores from the logs
Plain-agent baselines (2026)	Model-matched plain baselines before crediting architecture	We already report vanilla and handled side by side

Our own gap, from the code and the log: the only oracle that exists is the goal oracle (goal_source=oracle, used by C0). There is no grounding oracle and no plan oracle, and C0 is paired only with the classical pilot. The handle effort was also unequal: FULL got five rounds of fixes until it plateaued (D-176 to D-179), while C2-C8, C10 and C13 got one round each. So their handled scores are lower bounds, not capabilities.

What the field does for question 2

Benchmark	Design	What we take
EmbodiedBench (ICML 2025)	Six capability subsets, including long-horizon, spatial awareness and complex instructions	Group tasks by capability
LH-VLN (CVPR 2025)	Multi-stage tasks with subtask-level metrics (ISR, CSR, CGT)	Per-stage success, partial credit
LoTa-Bench (ICLR 2024)	Small models pass short tasks and fail longer ones	A task-length gradient
UAV-ON, OpenFly, CityNav, AeroVerse (2024-2025)	Aerial object-goal search, aerial VLN, landmark-relative city goals, spatial reasoning	Aerial task families

Our current set measured against this: basic and obstacle tasks are near 10/10 for most architectures, so they no longer separate them. Only 2 planning tasks discriminate (gate and ordered visit), and the other 2 are latency-bound (D-177), so they measure timing, not planning. There is no length gradient, no memory or revisit task, and no spatial-relation task.

Proposed experiments, ranked by effort against what they answer

#	Experiment	Answers	Effort	Recommendation
A	**Oracle swap per level**, on development seeds: (1) plan oracle: the correct typed program for each mission, written from the public mission text only; (2) grounding oracle: the true object box and range, given only when the object is in view (StructNav masking); (3) pilot: C0 with the VLA pilot (C0-VLA). Each swapped into each architecture that has that level	Which level limits each architecture: capability vs logic	Small to medium: three switches, privileged and never reported as results	**First**
B	**Equal handle budget**: every architecture gets the same number of diagnose-fix rounds (e.g. up to three, stopping after a round with no gain), and the per-round curve is reported	Whether each plateau is real, so "the architecture's ceiling" is a fair claim	Medium (GPU hours)	Second
C	**Latency-relaxed twins** of follow / closing (marker at 0.3 m/s, wall closing at 12 s)	Whether those zeros are planning or timing	Small: two task variants	With A
D	**Harder planning tasks**: ordered visits of 3 and 4 pillars (length gradient) with per-stage credit; condition + search ("if the blue pillar is not in view, search to the right"); revisit from memory ("go to red, then blue, then back to red", red out of view); a spatial relation ("the pillar next to the car")	Separates architectures where the current set saturates	Medium: new CapabilityEnv scenarios plus scripted fixtures	Third, after A shows where the limits are

A and C reuse the existing runtime, and their oracles feed only the run being diagnosed. Like C0, they are reference lines and never results.

Out of scope

- Importing the UAV benchmarks' datasets or simulators (OpenFly, CityNav, UAV-ON, AeroVerse): they need their own engines, and we borrow task designs only.
- Learned perception upgrades: the paper keeps one model and one runtime.
- Any new held-out claim on seeds 1-10; they are used (D-184). Seeds 11-40 remain.

Sources

- PRISM: https://arxiv.org/abs/2605.11534
- StructNav: https://arxiv.org/abs/2305.16925
- OPEx: https://arxiv.org/abs/2403.03017
- CanTest: https://arxiv.org/abs/2604.25161
- Embodied Agent Interface: https://arxiv.org/abs/2410.07166
- Plain-agent baselines: https://arxiv.org/abs/2607.13085
- Navigating to Objects in the Real World: https://arxiv.org/abs/2212.00922
- EmbodiedBench: https://arxiv.org/abs/2502.09560
- LH-VLN: https://arxiv.org/abs/2412.09082
- LoTa-Bench: https://arxiv.org/abs/2402.08178
- OpenFly: https://arxiv.org/abs/2502.18041
- UAV-ON: https://arxiv.org/abs/2508.00288
- CityNav: https://arxiv.org/abs/2406.14240
- AeroVerse: https://arxiv.org/abs/2408.15511

PDFs: papers/evaluation/ (README lists the mechanism taken from each).